

Esercitazione 5: Campionamento, decisione e sincronismo di pacchetto

EDOARDO NORI-0001123033

Copiare all'interno del file Es5PacketRX_RealTime_Es5.m il codice prodotto durante l'esercitazione 4. Rinominarlo in Es5PacketRX_RealTime_**cognome_nome**.m

In particolare, aggiungere i blocchi mancanti secondo le seguenti linee guida.

C) Campionamento a tempo di simbolo

Realizzare una funzione che dato il segnale A (ovvero $A(nT)$) ed il segnale synchwave (ovvero $s(nT)$) è in grado di produrre in output due vettori:

```
[softbits,sampinst] = campionatore(vettore,vettore)
```

- il vettore dei campioni A_k chiamato softbits
- il vettore che contiene gli istanti di campionamento, a cui sono stati estratti i campioni, di nome sampinst

```
Incollare qui il codice della funzione (font Courier size 8)

function [softbits,sampinst] = campionatore(A, synchwave)
% Ritorna il segnale A campionato sui fronti (+) di synchwave
% La synchwave viene traslata tra 0 e 1. Vengono poi individuati i fronti
% positivi o negativi mediante diff che effettua la differenza tra
% elementi adiacenti di un array. Con find vengono trovati gli istanti di
% campionamento, i quali sono poi passati come indici di A.
sample = [0, diff(synchwave > 0)];
sampinst = find(sample == 1);
softbits = A(sampinst);
end
```

D) Decisione

Sviluppare una funzione che realizza il decisore (comparatore a soglia). Il decisore sarà una funzione che legge il vettore softbits (ovvero i campioni A_k) e restituisce un vettore decoded (ovvero i bit b_k) i cui elementi sono "0" o "1".

```
Incollare qui il codice della funzione (font Courier size 8)

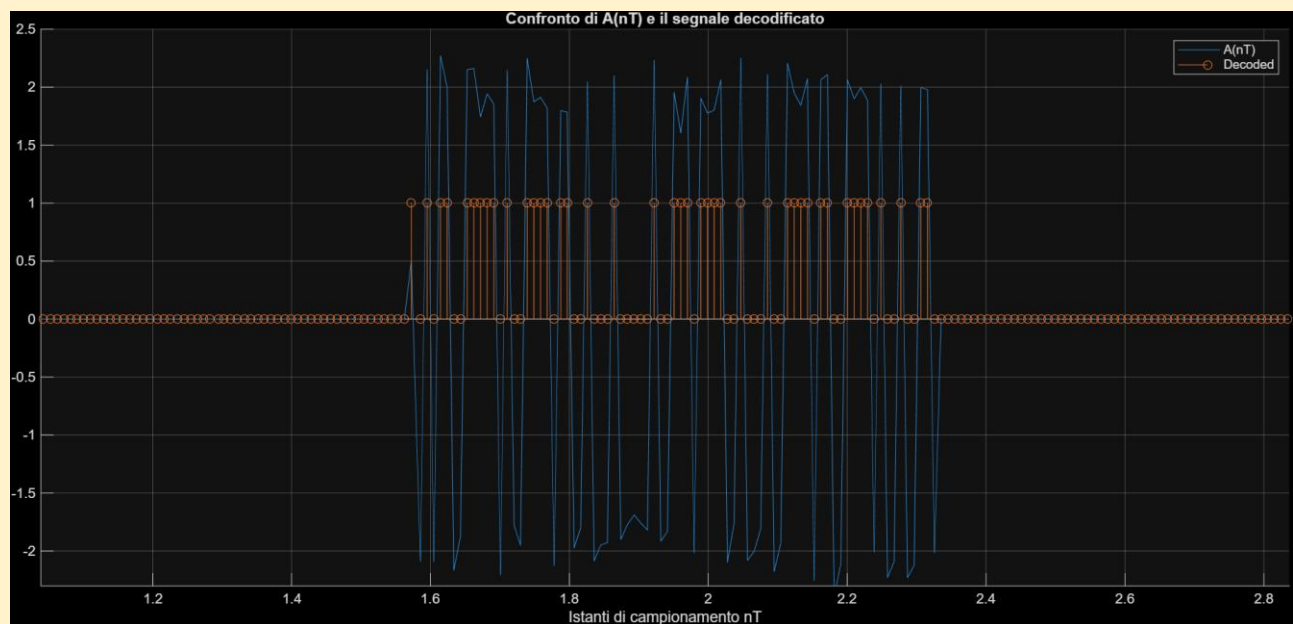
E = max(abs(IrM))^2 / (2*rb/5)*10^(rtlsdr_gain/10);
decoded = softbits > 0.8*E;

Commento:
Si è preferito non utilizzare un decisore a soglia zero bensì uno a soglia dinamica
basata sull'energia del segnale. Risulta importante ricordarsi che il dispositivo
rtlsdr ha un suo guadagno intrinseco del quale bisogna tenere conto. Il fatto 0.8 è
basato sull'osservazione empirica della simulazione. In questo modo è però possibile
ottenere un grafico più pulito e meno soggetto a rumore, quanto detto è osservabile
nelle schiere di zeri antecedenti e posteriori al pacchetto.
```

Grafico dopo il decisore

Produrre un unico grafico con il segnale $A(nT)$ (cioè A), ottenuto con plot(), ed il segnale decoded, ottenuto con stem().

Incollare qui sotto il grafico.



Controllare che ci sia corrispondenza tra il segnale demodulato e i bit decodificati!

E) Sincronismo di pacchetto

Nella sezione E dello script, aggiungere il codice necessario che implementa l'estrattore del sincronismo di pacchetto tramite la cascata delle operazioni di *mapping* e *filtraggio FIR*. Il correlatore deve produrre in uscita un vettore di nome `packsynch` corrispondente al segnale y_n nella corrispondente slide (che servirà nel successivo blocco F già predisposto).

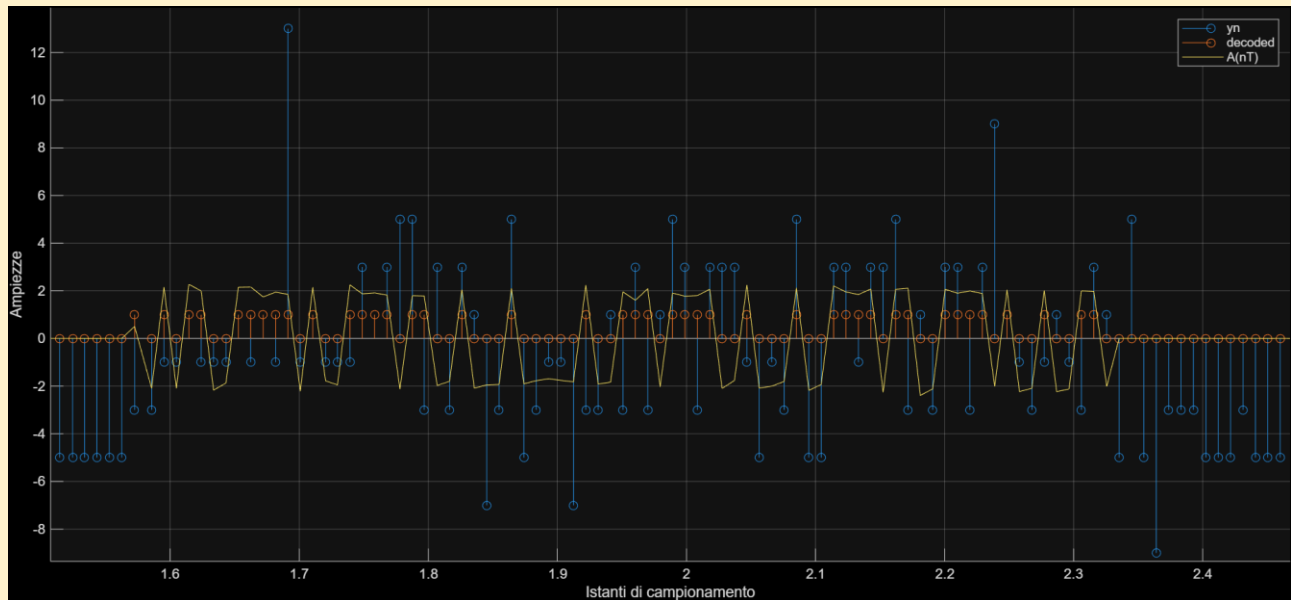
```
Incollare qui il codice che implementa il correlatore (font Courier size 8)

decoded_mapped = mapper(decoded, 1);
yn = filter(barker13_flip, 1, decoded_mapped);
packsynch = yn;
```

Analisi del sincronismo di pacchetto

Mettere in un **unico grafico** i vettori `packsynch` (ovvero y_n) e `decoded` (ossia b_k), graficati con `stem`, e il segnale A (ovvero $A(nT)$) graficato con `plot` e tratto continuo.

Incollare qui sotto il grafico.



Commentare il grafico: decoded è coerente con A(nT) come ci si aspetterebbe. Inoltre si può ben vedere che yn riconosce l'ID del pacchetto, dato l'impulso di valore 13 alla fine dell'ID.

F) Estrazione del payload, controllo di parità, conversione dei dati, visualizzazione a schermo

Rimuovere il comando `return` prima della sezione F dello script.

Rilanciare lo script per elaborare completamente il segnale ricevuto.

Si riporti qui sotto la stringa decodificata che appare nella command window (font Courier size 8)

Sensore 4: Temp=24.16°C; Hum=31.5%; Press=1013.58hPa; Batt=4.05V;

FACOLTATIVO:

Decomentare la parte di codice che permette l'acquisizione del segnale da chiavetta e osservare i pacchetti ricevuti in tempo reale.

Istruzioni:

1. Grafici e risultati prodotti devono recare sempre le relative unità di misura (ms, Hz).
2. Fare le opportune sostituzioni nelle parti evidenziate in verde prima della consegna.
3. Rinominare questo file sostituendo nome e cognome.
4. Caricare su Virtuale entro la scadenza comunicata a lezione:
 - questa relazione compilata in ogni sua parte in **FORMATO PDF**
 - tutto il codice sviluppato (script, funzioni) e i file generati (.zip o .rar)
 - il file PacketRX_es5_cognome_nome.m

Attenzione: non usare **MAI** la funzione "screenshot" del vostro SO. Salvare **SEMPRE** l'immagine con il comando `print()` posizionato dopo il `plot()` con una risoluzione di almeno 300dpi (uno qualunque di questi formati va bene):

```
print(gcf, 'nomefile.png', '-dpng', '-r300');  
print(gcf, 'nomefile.jpg', '-djpeg', '-r300');  
print(gcf, 'nomefile.pdf', '-dpdf', '-r300');
```

oppure, ancora meglio, in formato vettoriale come l'Encapsulated PostScript (EPS):

```
print(gcf, 'nomefile.eps', '-deps');
```

Good practice

Comment your code; describe each module with the following information:

- *Name and purpose of the Module*
- *Description of the Module (in brief)*
- *Example of usage: input and output*
- *Author, date and version*

Example in Matlab:

```
function [mymod] = mymodexp(b,e,n)

    % mymodexp(b,e,n) gives  $b^e \bmod n$ , where b, e, n are integers
    %
    % iterative implementation based on the property
    %  $(a \cdot (a^m \bmod n)) \bmod n = a^{(m+1)} \bmod n$  to avoid numerical overflow
    %
    % example of usage: mymodexp(137,56,14731) gives as output 6849
    %
    % Andrea Giorgetti, Oct. 5, 2020, version 1.2 %
```
